GUIA VISUAL 038

PIPX, FLATPAK E APPIMAGE

`externally-managed-environment` *não é obstáculo a contornar — é a proteção funcionando.*

038
GUIAS LINUX

PYTHON: TRÊS FORMAS DE INSTALAR, TRÊS ISOLAMENTOS

`sudo pip install`

Escreve em `/usr/lib/python3/`, o mesmo lugar dos pacotes `.deb` — e sem consultar o `dpkg`, que passa a **mentir sobre as versões**.

Uma biblioteca serve a todos os programas: um conflito quebra vários.

BLOQUEADO PELA PEP 668

`python3 -m venv`

Um diretório por **projeto**, com seu próprio Python e suas bibliotecas. Isolamento completo; nada toca o sistema.

Exige `source .venv/bin/activate` antes de usar; remover é `rm -rf .venv`.

IDEAL PARA DESENVOLVER

`pipx install`

Um ambiente virtual por **aplicação**, criado e gerido sozinho. Os executáveis aparecem em `~/.local/bin`, já no `PATH`.

Usar não exige ativar nada. Duas ferramentas com bibliotecas incompatíveis convivem sem se ver.

IDEAL PARA INSTALAR FERRAMENTAS

A confusão é entre as duas da direita: o `venv` serve a quem *escreve* Python; o `pipx` a quem só quer *usar* a ferramenta. Por baixo o pipx cria um venv — ele apenas cuida disso por você. Comandos que resolvem o dia a dia: `pipx run <ferramenta>` executa e descarta (uso único), `pipx install git+https://…` instala direto do repositório do autor sem clonar, `pipx upgrade-all` e `pipx reinstall-all` (depois de o Python do sistema mudar de versão).

A ESCADA DA DECISÃO: DO INTEGRADO AO INVISÍVEL

1. **Está no repositório do Kali?** → `sudo apt install` — sempre a primeira opção: registrado no `dpkg` e atualizado sozinho.
2. **É uma ferramenta Python?** → `pipx install` — ou `pipx run` se o uso for único.
3. **É Go, Rust ou Node?** → `go install` · `cargo install` · `npm` com prefixo local. Binário em `~/go/bin`, `~/.cargo/bin`, `~/.local/bin` — ponha no `PATH`.
4. **Tem imagem de contêiner oficial?** → `docker run --rm` — o isolamento mais forte e a remoção mais completa; o custo é mapear volumes.
5. **É aplicação gráfica de desktop?** → `flatpak install` — conferindo antes o `flatpak info --show-permissions`.
6. **Existe AppImage do autor?** → `chmod +x` só **depois** de conferir o hash e de `--appimage-extract` para inspecionar.
7. **Nenhuma das anteriores** → `compilar` — e empacotar com `checkinstall` para não virar software invisível.

▼ SE NÃO, DESÇA UM DEGRAU — CADA DEGRAU ACRESCENTA ALGO QUE SÓ VOCÊ VAI LEMBRAR DE ATUALIZAR

SNAP

Imagem `squashfs` + daemon `snapd` residente, confinado por AppArmor; atualiza sozinho. Não vem no Kali.

FLATPAK

Aplicação + *runtime* compartilhado, *sandbox* por namespaces com *portals*. Atualiza sob comando; o 1º app baixa um runtime grande.

APPIMAGE

Arquivo único executável, nada instalado. **Isolamento nenhum**; atualização e registro, nenhum.

Dois riscos que o nome já entrega. (1) `pip install --break-system-packages` faz exatamente o que diz: é aceitável em contêiner descartável ou imagem de build, mas numa máquina de trabalho produz um Python em que o `dpkg` afirma uma versão e o disco tem outra — e o sintoma aparece semanas depois, num `apt` que falha ao importar um módulo. Se o impulso for usá-la, o comando certo é `pipx install`. **(2)** Um **AppImage roda sem confinamento algum**, com todo o poder do seu usuário: `~/.ssh`, o chaveiro, os arquivos do cliente. `--appimage-extract` e a verificação do hash publicado pelo autor são a única defesa **antes** da primeira execução. Vale o mesmo para Flatpak que pede `--filesystem=home`: isso anula boa parte do isolamento — `flatpak override --nofilesystem=home` corrige.

RESUMO RÁPIDO

- ✓ O bloqueio é um arquivo real: `/usr/lib/python3*/EXTERNALLY-MANAGED`. Dentro de um `venv` ativado o `pip` funciona normalmente.

NO KALI

Rode `apt search --names-only` antes de qualquer `pipx install`: o Kali empacota centenas de ferramentas Python como `.deb`, às vezes com nome diferente do projeto no